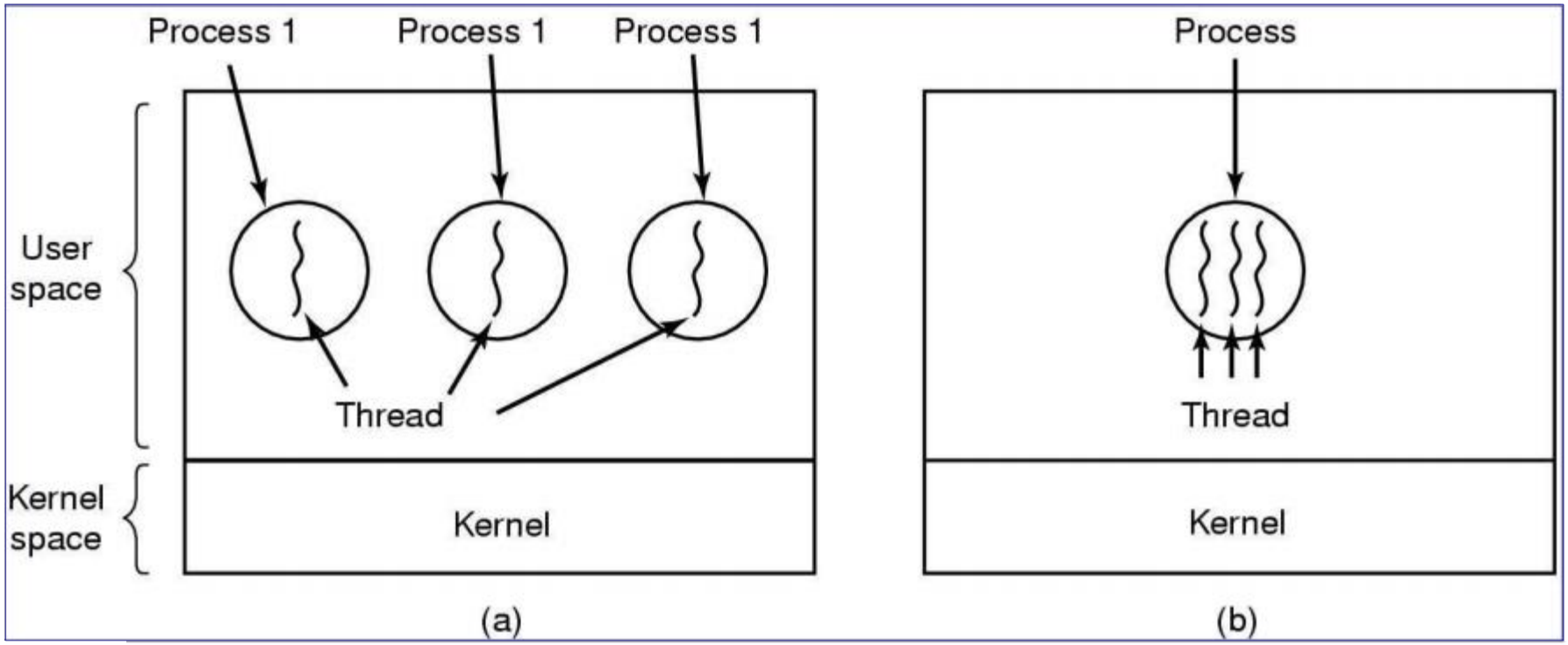
What is Thread?

Threads, like process, are a mechanism to allow a program to do more than one thing at a time.

Conceptually, a thread (also called *lightweight process*) exists within a process (heavyweight process). Threads are a finer-grained unit of execution than processes

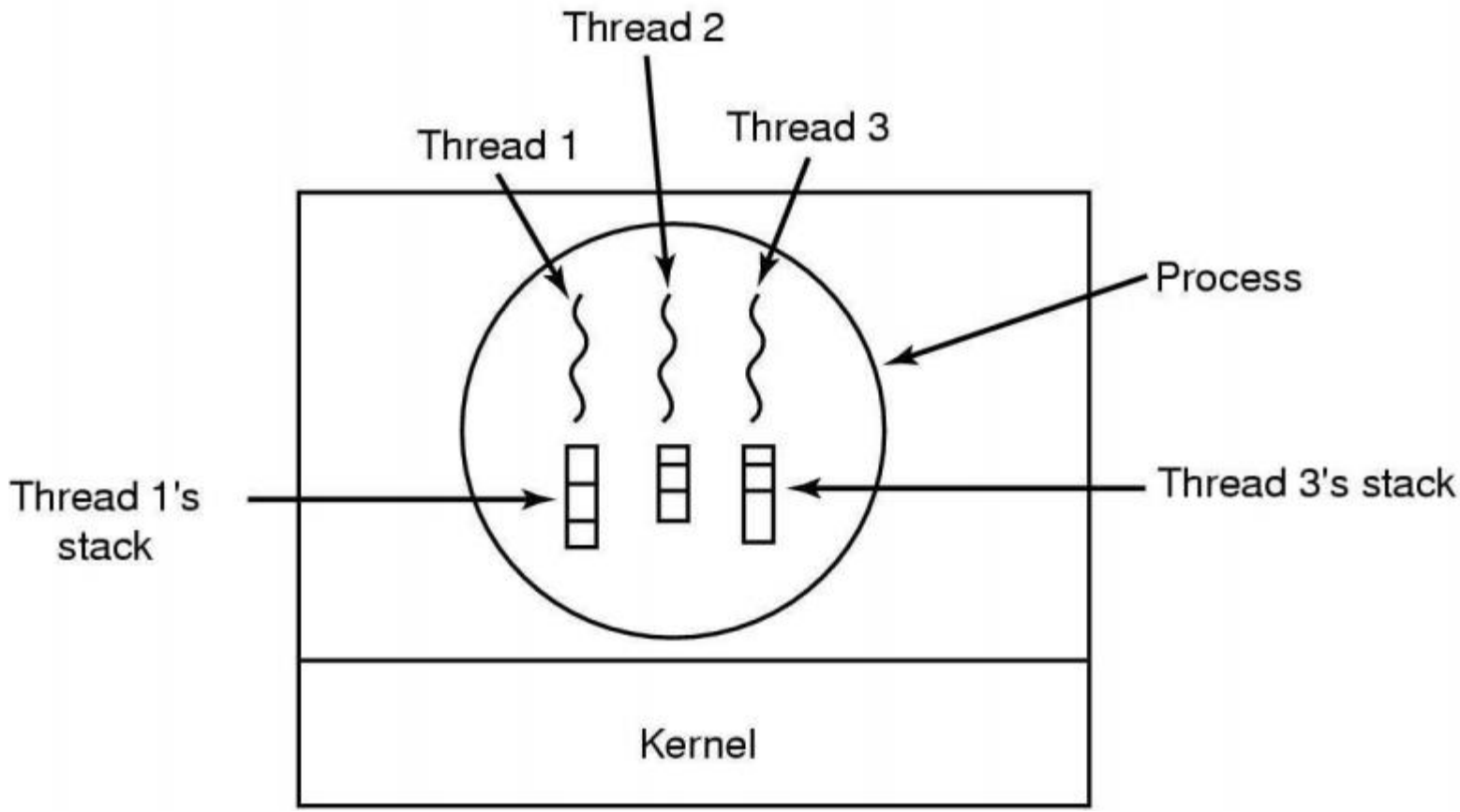
- Traditional threads has its own address space and a single thread of control.
- The term multithreading is used to describe the situation of allowing the multiple threads in same process.
- When multithreaded process is run on a single-CPU system, the threads take turns running as in the multiple processes.
- All threads share the same address space, global variables, set of open file, child processes, alarms and signals etc.



(a) Three process each with one thread (b) one process with three threads.

- *Figure (a)* organization is used when three processes are unrelated whereas *(b)* would be appropriate when the three threads are actually part of the same job and are actively and closely cooperating with each other.

- Each thread maintain its own stack.



Example – Word processor

Needs to accept user input, display it on screen, spell check, auto save and grammar check.

- Implicit: Write code that reads user input, displays/formats it on screen, calls spell checked etc. while making sure that interactive response does not suffer.
- Threads: Use threads to perform each task and communicate using queues and shared data structures
- Processes: expensive to create and do not share data structures and so explicitly passed.

Others: Spreadsheet, Server for www, browser etc.

Advantages:

- Responsiveness.
- Resource sharing.
- Economy.

Problems: designing complexity.

Users and Kernel Threads

User Threads (created by user level programs):

Thread management done by user-level threads library.

- Implemented as a library
- Library provides support for thread creation, scheduling and management with no support from the kernel.
- Fast to create
- If kernel is single threaded, blocking system calls will cause the entire process to block.

Example: POSIX Pthreads, Mach C-threads.

Kernel Threads (created by kernel):

Supported by the Kernel

Kernel performs thread creation, scheduling and management in kernel space.

Slower to create and manage

Blocking system calls are no problem

Most OS's support these threads

Examples: WinX, Linux

Task

- Differentiate between process and thread.

Benefits of Multithreading

- **Improved throughput**: Many concurrent compute operations and I/O requests within a single process.
- Simultaneous and fully symmetric use of **multiple processors** for computation and I/O.
- **Superior application responsiveness**: If a request can be launched on its own thread, applications do not freeze. An entire application will not block, or otherwise wait, pending the completion of another request.
- **Improved server responsiveness:** Large or complex requests or slow clients don't block other requests for service. The overall throughput of the server is much greater.

- **Minimized system resource usage:** Threads impose minimal impact on system resources. Threads require less overhead to create, maintain, and manage than a traditional process.
- **Program structure simplification**: Threads can be used to simplify the structure of complex applications, such as server-class and multimedia applications. Simple routines can be written for each activity, making complex programs easier to design and code, and more adaptive to a wide variation in user demands.
- **Better communication**: Thread synchronization functions can be used to provide enhanced process-to-process communication. In addition, sharing large amounts of data through separate threads of execution within the same address space provides extremely high-bandwidth, low-latency communication between separate tasks within an application.

Multithreading Models

Most multithreading models fall into one of the following categories of threading implementation:

- Many-to-One
- One-to-One
- Many-to-Many

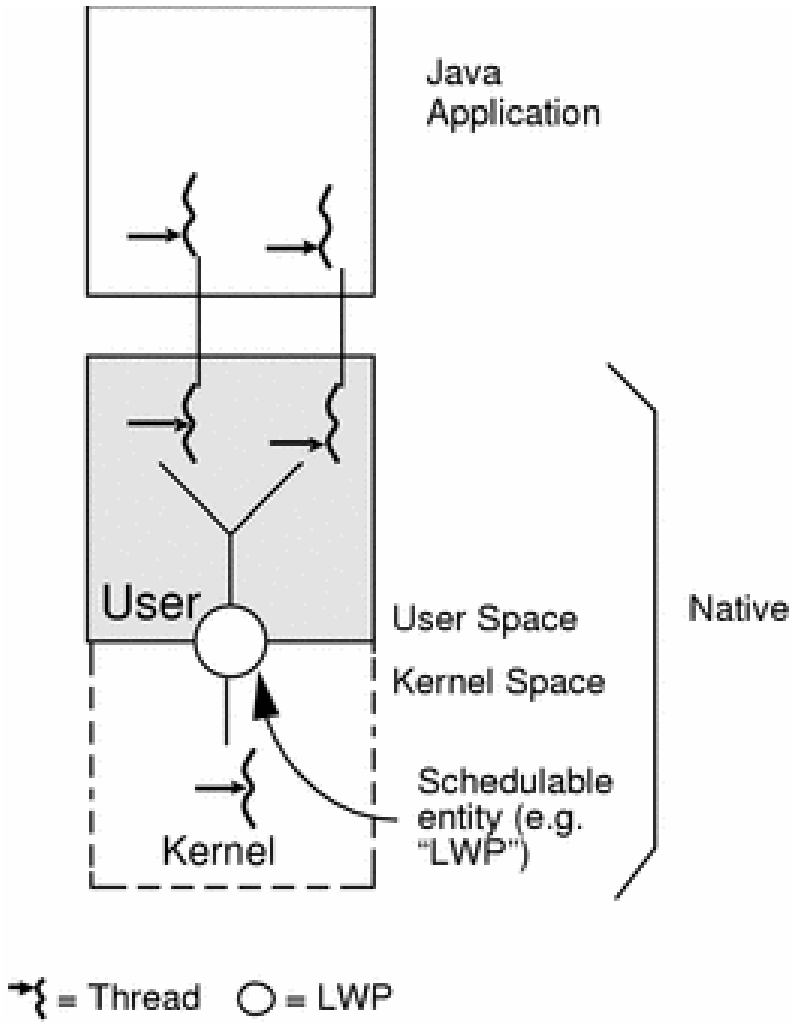
Many-to-One Model (Green Threads)

Implementations of the many-to-one model (many user threads to one kernel thread) allow the application to create any number of threads that can execute concurrently.

In a many-to-one (user-level threads) implementation, all threads activity is restricted to user space. Additionally, only one thread at a time can access the kernel, so only one schedulable entity is known to the operating system. As a result, this multithreading model provides limited concurrency and does not exploit multiprocessors.

The initial implementation of Java threads on the Solaris system was many-to-one, as shown in the following figure.

Many-to-One Model (Green Threads)



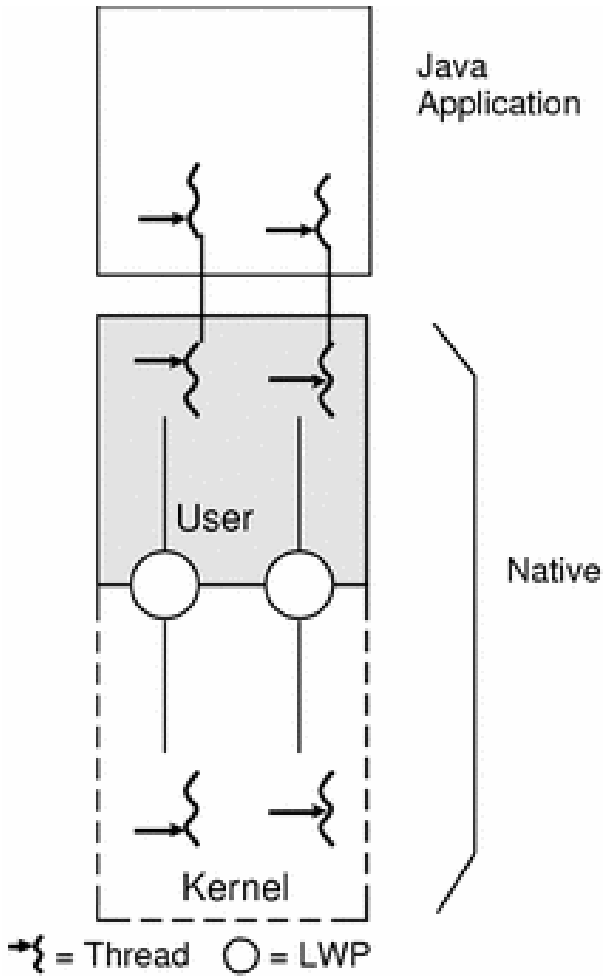
One-to-One Model

The one-to-one model (one user thread to one kernel thread) is among the earliest implementations of true multithreading. In this implementation, each user-level thread created by the application is known to the kernel, and all threads can access the kernel at the same time.

The main problem with this model is that it places a restriction on you to be careful with threads, as each additional thread adds more "weight" to the process.

Consequently, many implementations of this model, such as Windows NT and the OS/2 threads package, limit the number of threads supported on the system.

One-to-One Model



Many-to-Many Model

- The many-to-many model (many user-level threads to many kernel-level threads) avoids many of the limitations of the one-to-one model, while extending multithreading capabilities even further.
- The many-to-many model, also called the two-level model, minimizes programming effort while reducing the cost and weight of each thread.
- In the many-to-many model, a program can have as many threads as are appropriate without making the process too heavy or burdensome.

Many-to-Many Model

- In this model, a user-level threads library provides sophisticated scheduling of user-level threads above kernel threads. The kernel needs to manage only the threads that are currently active.
- A many-to-many implementation at the user level reduces programming effort as it lifts restrictions on the number of threads that can be effectively used in an application.
- A many-to-many multithreading implementation thus provides a standard interface, a simpler programming model, and optimal performance for each process.
- The Java on Solaris operating environment is the first many-to-many commercial implementation of Java on an Multithreading operating system.

Many-to-Many Model